《算法分析与设计》

课后作业

学　　号　2024112639

姓　　名　毛露涵

年　　级　2024 级

专　　业　软件工程

院　　系　计算机与人工智能学院

第一大题（30%）	第二大题（20%）	第三大题（50%）	第四大题（0%）	总分	批阅人签字

2025-2026 学年第 2 学期

第 7 章 回溯法和分支限界法

一、题目 1（数独游戏）（共 30 分）

数独游戏是在 9*9 的方格中填放 1~9 的数字，要求每一行、每一列以及 3*3 的方格中的数字均不能相同，如下图所示。

1	4	3	6	2	8	5	7	9
5	7	2	1	3	9	4	6	8
9	8	6	7	5	4	2	3	1
3	9	1	5	4	2	7	8	6
4	6	8	9	1	7	3	5	2
7	2	5	8	6	3	9	1	4
2	3	7	4	8	1	6	9	5
6	1	9	2	7	5	8	4	3
8	5	4	3	9	6	1	2	7

现方格中某些数字为空缺（用 0 表示），希望你编写程序能够将空缺的数字补齐。

输入要求：

输入包含 9 行，每一行包含 9 个数字，对应每个方格中的数字，0 表示该方格的数字为空。

输出要求：

输出为 9 行，每行 9 个数字，也就是补齐后的 9*9 的方格中的数字。

样例输入：

103000509

002109400

000704000

300502006

060000050

700803004

000401000

009205800

804000107

样例输出：

143628579

572139468

986754231

391542786

468917352

725863914

237481695

619275843

854396127

要求：

1、写出问题的目标函数和约束函数。（10 分）

设数独棋盘为：X={xij | 1≤i≤9，1≤j≤9}

其中 xij 表示第 i 行第 j 列填入的数字。

本题的目标不是求最大值或最小值，而是求一个满足全部条件的可行解。目标函数可以表示为：F(X)=ΣΣI(xij≠0)

目标是使：F(X)=81

即 9×9 的所有空格都被填满。

约束函数如下：

（1）数值范围约束：

1≤xij≤9

（2）已知数字不能改变：

若原棋盘中 aij≠0，则 xij=aij。

（3）行约束：

同一行中的 9 个数字互不相同。

（4）列约束：

同一列中的 9 个数字互不相同。

（5）3×3 小方格约束：

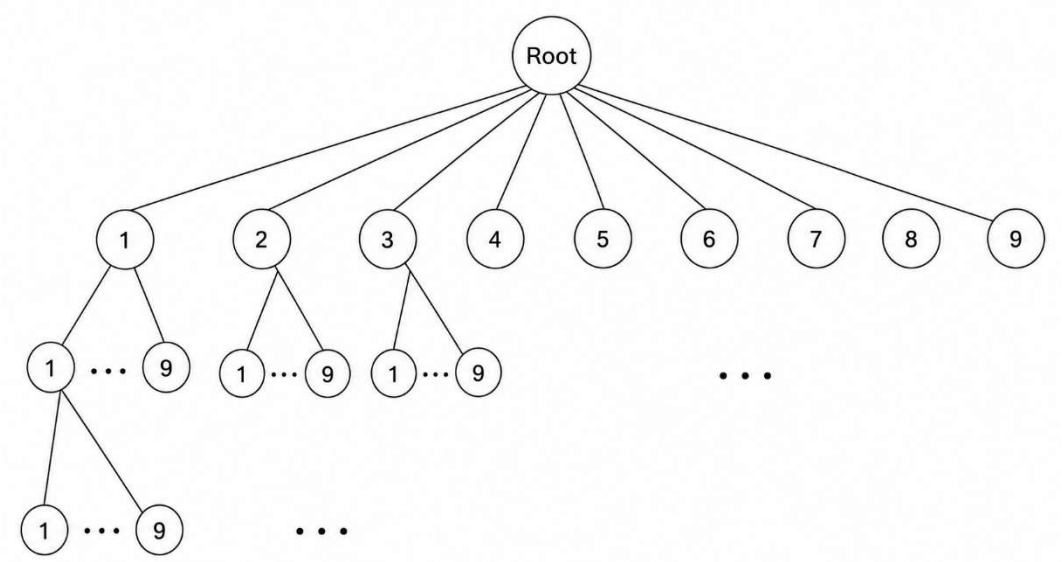
每一个 3×3 小方格中的 9 个数字互不相同。

在回溯搜索中，约束函数可以写为：

C(i,j,v)=true，表示数字 v 在第 i 行、第 j 列和所在 3×3 小方格中均没有出现，可以填入。

C(i,j,v)=false，表示数字 v 与已有数字冲突，不能填入。

2、画出该问题的解空间树（标出解空间树的宽度和深度等于什么。如果树的规模太大，可以用省略号）。（10 分）



样例输入中共有 51 个空格，因此该解空间树的深度为 51。每个空格最多可以尝试 1～9 共 9 个数字，所以每个结点的最大分支数为 9。第 k 层最多有 9^k 个结点。实际搜索时，如果某个数字违反行、列或 3×3 小方格约束，就立即剪枝，所以实际搜索结点数小于完整的 9^51。

3、采用回溯法编写程序实现上述题目要求，附源代码，并给出由程序算出的上述测试用例的输出，附屏幕截图。（10 分）

```
#define _CRT_SECURE_NO_WARNINGS
#include <bits/stdc++.h>
using namespace std;

int a[9][9];
bool rowUse[9][10];
bool colUse[9][10];
bool boxUse[9][10];

int getBox(int x, int y) {
    return (x / 3) * 3 + y / 3;
}

bool dfs(int pos) {
    if (pos == 81) return true;

    int x = pos / 9;
    int y = pos % 9;

    if (a[x][y] != 0) {
```

```
            return dfs(pos + 1);
        }

        int b = getBox(x, y);

        for (int v = 1; v <= 9; v++) {
            if (!rowUse[x][v] && !colUse[y][v] && !boxUse[b][v]) {
                a[x][y] = v;
                rowUse[x][v] = true;
                colUse[y][v] = true;
                boxUse[b][v] = true;

                if (dfs(pos + 1)) return true;

                a[x][y] = 0;
                rowUse[x][v] = false;
                colUse[y][v] = false;
                boxUse[b][v] = false;
            }
        }

        return false;
}

int main() {
        string s;

        for (int i = 0; i < 9; i++) {
            cin >> s;
            for (int j = 0; j < 9; j++) {
                a[i][j] = s[j] - '0';

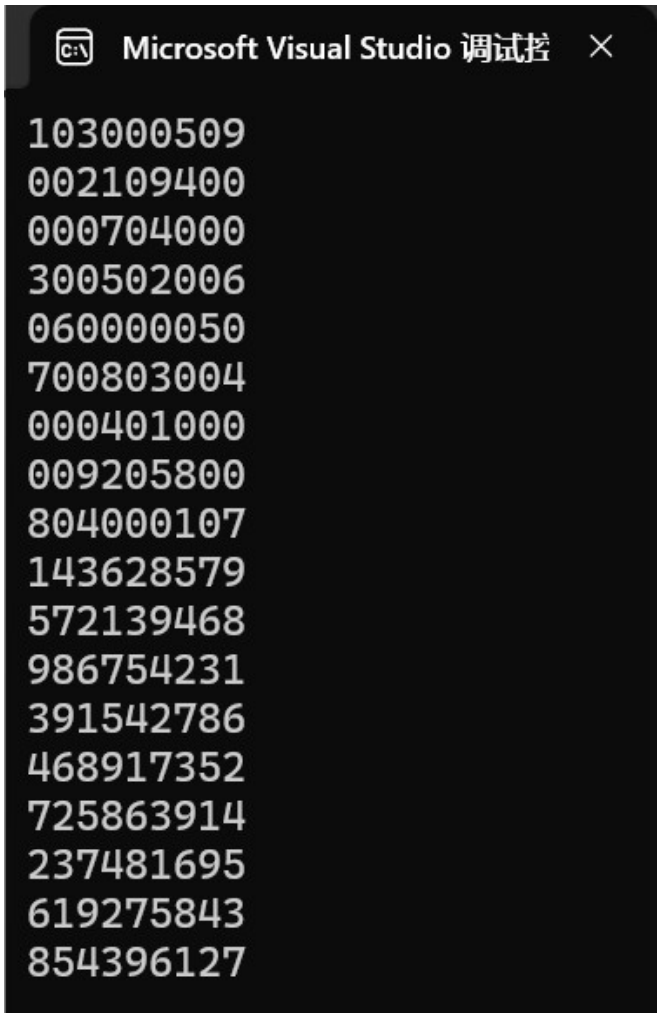
                if (a[i][j] != 0) {
                    int v = a[i][j];
                    int b = getBox(i, j);

                    rowUse[i][v] = true;
                    colUse[j][v] = true;
                    boxUse[b][v] = true;
                }
            }
        }

        dfs(0);
```

```
    for (int i = 0; i < 9; i++) {
        for (int j = 0; j < 9; j++) {
            cout << a[i][j];
        }
        cout << endl;
    }

    return 0;
}
```



二、题目 2（最小步数，广度优先）（共 20 分）

一个人在一个由方格组成的矩形区域中按上、下、左、右四个方向移动（每次移动记为一步），区域内可能存在障碍物，每个障碍物占用一个方格，无法通行。计算人从起点方格移动终点方格所需的最少步数。

输入要求：

测试数据的第一行包含 7 个整数 n、 m、k、x0、y0、x1、y1，分别表示

矩形区域的行、列方格数（n 和 m），区域内障碍物的数量（k），人的起点方格坐标（行列号，x0 和 y0），终点方格坐标（行列号，x1 和 y1）。之后的 k 行，每行包含两个整数，分别表示障碍物所占用的方格的行列号。

输出要求：

输出 1 行。如果能够到达为所需的最少步数，否则输出 Impossible。

要求：

1、采用标准的广度优先算法编写程序实现上述题目要求（20 分）。

```
#define _CRT_SECURE_NO_WARNINGS
#include <bits/stdc++.h>
using namespace std;

struct Node {
    int x;
    int y;
    int step;
};

int n, m, k;
int sx, sy, ex, ey;
bool obs[105][105];
bool vis[105][105];

int dx[4] = { -1, 1, 0, 0 };
int dy[4] = { 0, 0, -1, 1 };

int bfs() {
    queue<Node> q;

    if (sx < 1 || sx > n || sy < 1 || sy > m) return -1;
    if (ex < 1 || ex > n || ey < 1 || ey > m) return -1;
    if (obs[sx][sy] || obs[ex][ey]) return -1;

    Node st;
    st.x = sx;
    st.y = sy;
```

```
    st.step = 0;

    q.push(st);
    vis[sx][sy] = true;

    while (!q.empty()) {
        Node cur = q.front();
        q.pop();

        if (cur.x == ex && cur.y == ey) {
            return cur.step;
        }

        for (int i = 0; i < 4; i++) {
            int nx = cur.x + dx[i];
            int ny = cur.y + dy[i];

            if (nx < 1 || nx > n || ny < 1 || ny > m) continue;
            if (obs[nx][ny] || vis[nx][ny]) continue;

            Node nxt;
            nxt.x = nx;
            nxt.y = ny;
            nxt.step = cur.step + 1;

            vis[nx][ny] = true;
            q.push(nxt);
        }
    }

    return -1;
}

int main() {
```

```
    cin >> n >> m >> k >> sx >> sy >> ex >> ey;

    for (int i = 0; i < k; i++) {
        int x, y;
        cin >> x >> y;
        obs[x][y] = true;
    }

    int ans = bfs();

    if (ans == -1) {
        cout << "Impossible" << endl;
    }
    else {
        cout << ans << endl;
    }

    return 0;
}
```

测试输入：

3 3 1 1 1 3 3

2 2

测试输出：

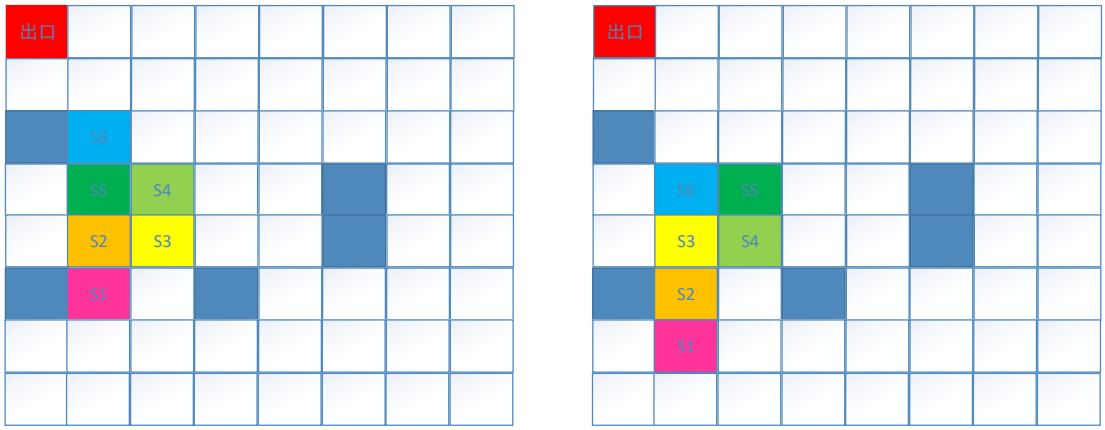
4



三、题目三（贪吃蛇，分支限界法）（共 50 分）

可能有不少的同学玩过“贪吃蛇“的游戏，游戏中蛇头带动整个蛇的移动，蛇身将沿蛇头移动过的位置进行移动。如果将整条蛇分成若干个方格，则可以表

示为 S1,S2,S3，…，SL，其中 S1 为蛇头，SL 为蛇尾，中间则是蛇头到蛇尾之间的部分。蛇在某个区域内移动时，如果蛇头所在位置的上下左右四个方向没有其它的物体，则蛇头 S1 可以朝其中任何一个方格移动，蛇身则填补前面移动后的区域，也就是 S2 移动到 S1 所在区域，S3 移动到 S2 所在区域……，依次类推，如下图所示。



备注：左图为蛇移动前的情况。右图为蛇向下移动一个方格后的情况。蓝色填充的方格表示有障碍物。

请判断蛇是否可以从它当前所在的位置移动到出口所在的位置，如果能，则给出最少需要移动的步数。如果不能，则输出“Impossible”。

输入要求：

输入的第 1 行包含三个整数 n, m（l<=n, m<=20） 和 L （2<=L<=8），分别表示蛇所在区域的大小（行，列方格数）以及蛇的长度。其后的 L 行用于表示从蛇头到蛇尾依次占用的方格的行列值。紧接着的 1 行包含一个整数 K，表示该区域内障碍物占用的方格数。之后的 K 行，每行包含两个整数，分别表示障碍物所占用的方格的行列值。出口所在的位置为（1，1），且出口处没有障碍物。

输出要求：

输出 1 行，如果蛇能够到达出口，则输出蛇从当前位置移动到出口位置最少需要移动的方格数量，否则输出“Impossible”。

样例输入：

5 5 4

5 3

4 3

4 2

3 2

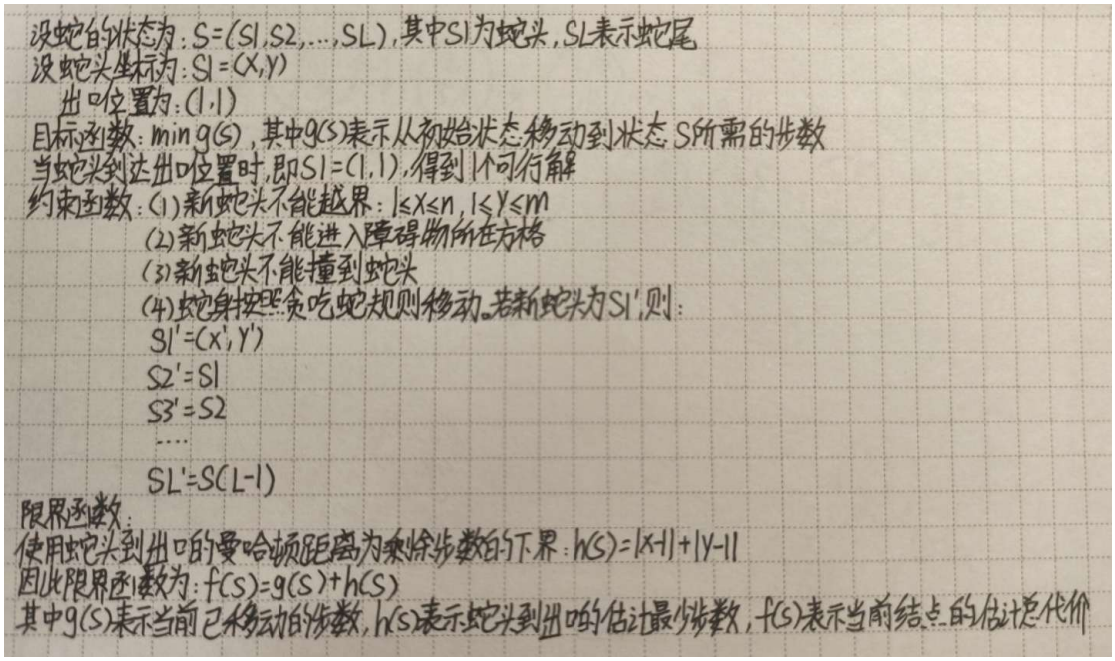
2

3 1

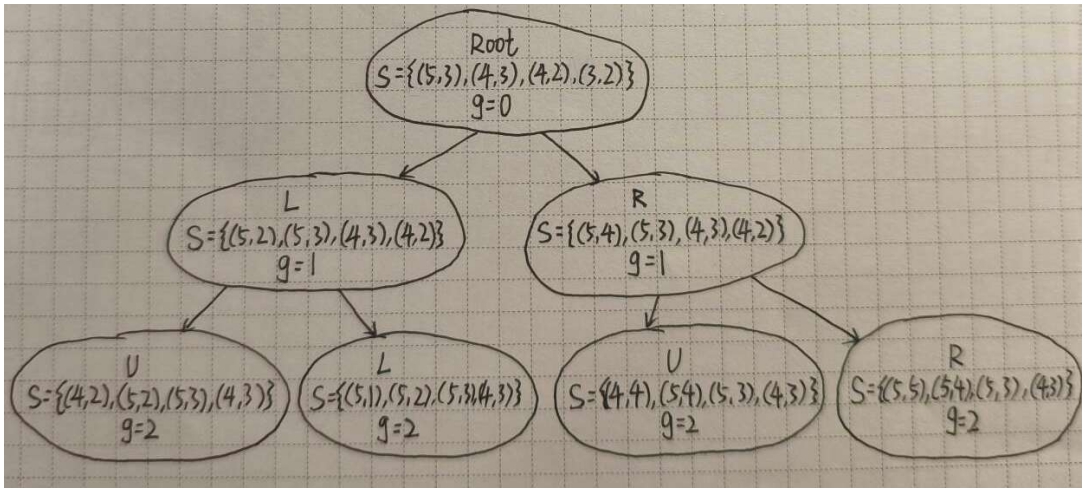
3 4

要求：

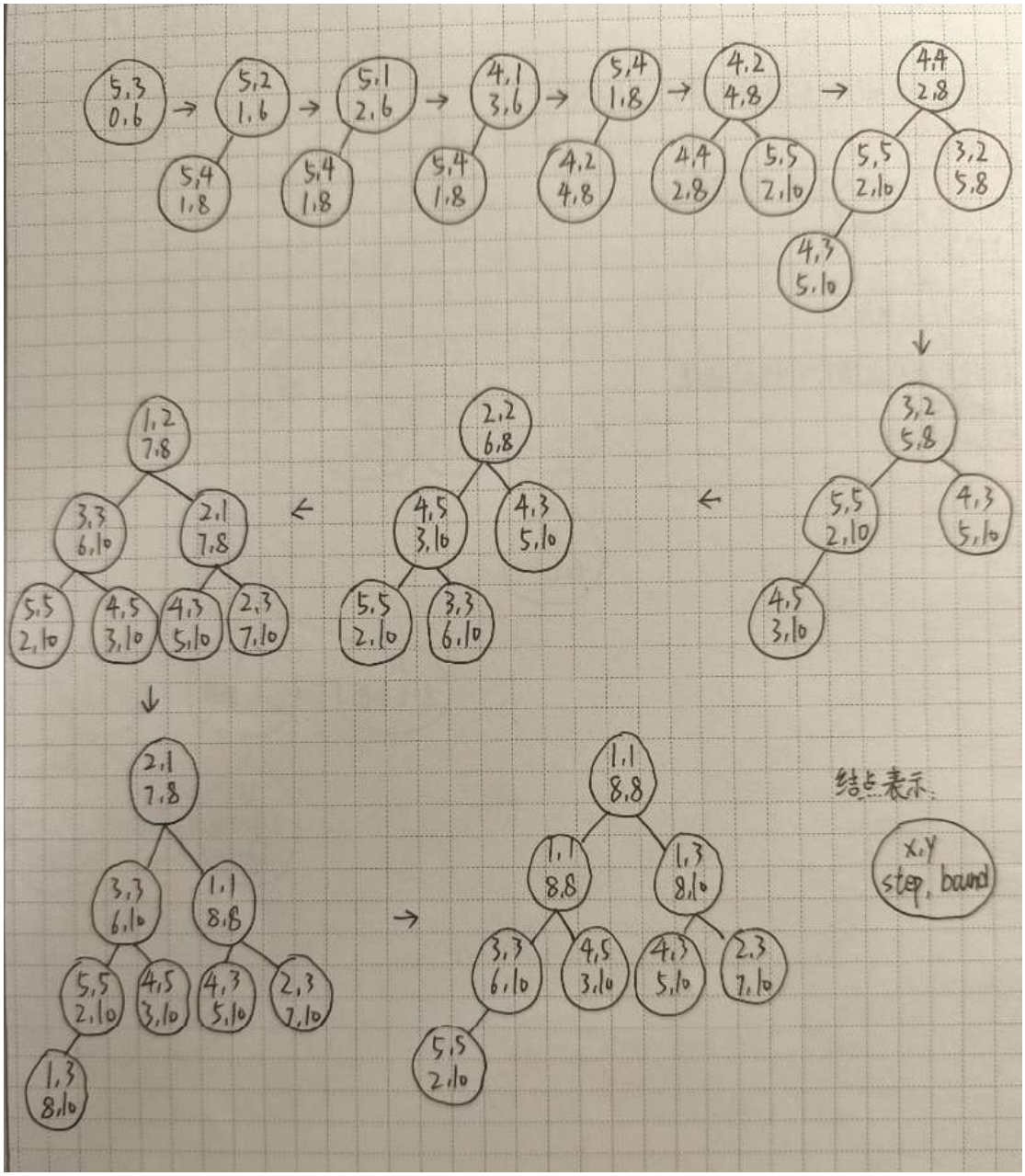
1、写出问题的目标函数、约束函数和限界函数。（10 分）



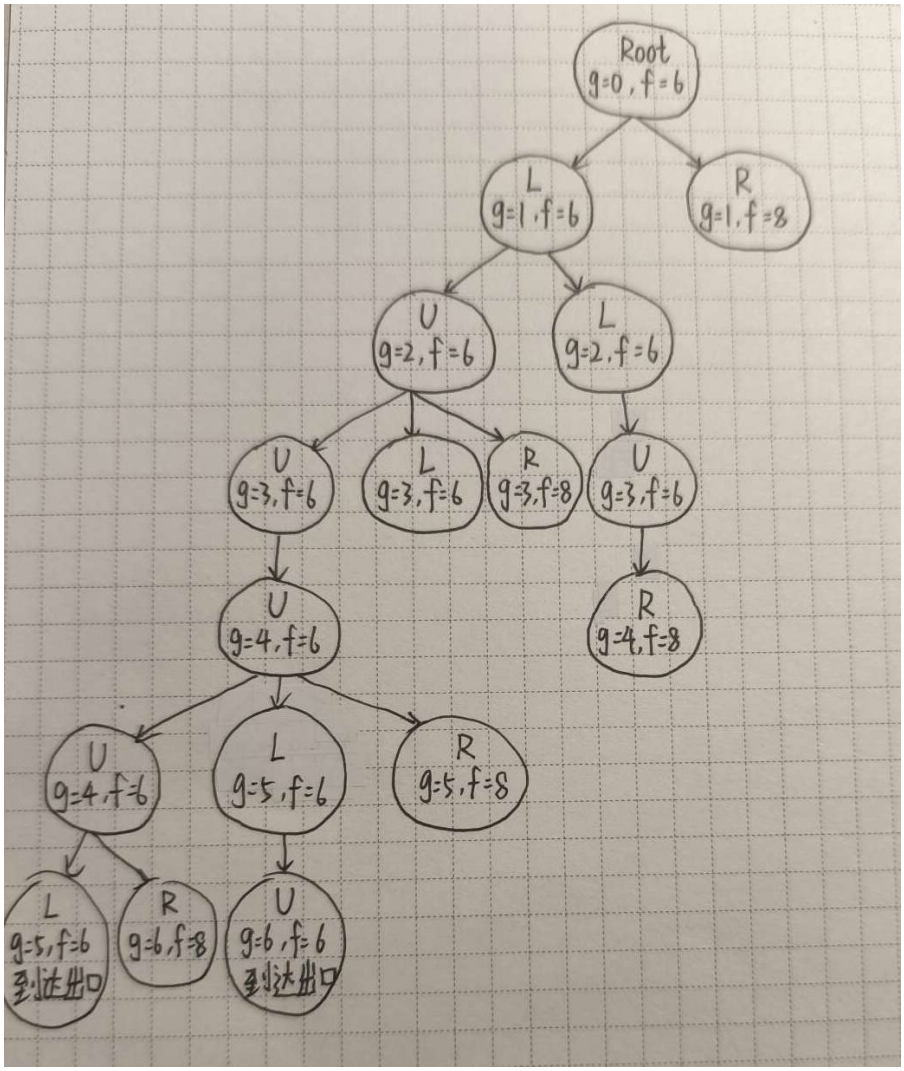
2、画出该问题的解空间树（只画 2 层结点，其中根结点是第 0 层）。（10 分）



3、画出用分支限界法求解该问题时堆的变化过程，指明堆中每个结点的变量值及其含义（参考课堂练习定义结点变量）。（10 分）



4、画出采用分支限界法求解该问题时的搜索空间树。（10 分）



5、采用分支限界法编写程序实现上述题目要求，附源代码，并给出由程序算出的上述测试用例的输出，附屏幕截图。（10 分）

```
#define _CRT_SECURE_NO_WARNINGS
#include <bits/stdc++.h>
using namespace std;

struct Node {
    vector<pair<int, int> > sn;
    int g;
    int f;
    int id;
};
```

```
struct Cmp {
    bool operator()(const Node& a, const Node& b) const {
        if (a.f != b.f) return a.f > b.f;
        if (a.g != b.g) return a.g > b.g;
        return a.id > b.id;
    }
};

int n, m, len;
bool obs[25][25];

int dx[4] = { -1, 1, 0, 0 };
int dy[4] = { 0, 0, -1, 1 };

int getH(const vector<pair<int, int> >& sn) {
    int x = sn[0].first;
    int y = sn[0].second;
    return abs(x - 1) + abs(y - 1);
}

string makeKey(const vector<pair<int, int> >& sn) {
    string s;

    for (int i = 0; i < (int)sn.size(); i++) {
        s.push_back((char)sn[i].first);
        s.push_back((char)sn[i].second);
    }

    return s;
}

bool inBody(const vector<pair<int, int> >& sn, int x, int y) {
    for (int i = 0; i < (int)sn.size() - 1; i++) {
        if (sn[i].first == x && sn[i].second == y) {
```

```
                return true;
            }
        }

        return false;
    }

    int solve(vector<pair<int, int> > st) {
        priority_queue<Node, vector<Node>, Cmp> pq;
        map<string, int> dist;

        Node root;
        root.sn = st;
        root.g = 0;
        root.f = getH(st);
        root.id = 0;

        pq.push(root);
        dist[makeKey(st)] = 0;

        int cnt = 0;

        while (!pq.empty()) {
            Node cur = pq.top();
            pq.pop();

            string key = makeKey(cur.sn);

            if (cur.g != dist[key]) continue;

            int hx = cur.sn[0].first;
            int hy = cur.sn[0].second;

            if (hx == 1 && hy == 1) {
```

```
            return cur.g;
        }

        for (int i = 0; i < 4; i++) {
            int nx = hx + dx[i];
            int ny = hy + dy[i];

            if (nx < 1 || nx > n || ny < 1 || ny > m) continue;
            if (obs[nx][ny]) continue;
            if (inBody(cur.sn, nx, ny)) continue;

            vector<pair<int, int> > nxt;
            nxt.push_back(make_pair(nx, ny));

            for (int j = 0; j < len - 1; j++) {
                nxt.push_back(cur.sn[j]);
            }

            int ng = cur.g + 1;
            string nk = makeKey(nxt);

            if (!dist.count(nk) || ng < dist[nk]) {
                dist[nk] = ng;

                Node nd;
                nd.sn = nxt;
                nd.g = ng;
                nd.f = ng + getH(nxt);
                nd.id = ++cnt;

                pq.push(nd);
            }
        }
    }
```

```
    return -1;
}

int main() {
    cin >> n >> m >> len;

    vector<pair<int, int> > st;

    for (int i = 0; i < len; i++) {
        int x, y;
        cin >> x >> y;
        st.push_back(make_pair(x, y));
    }

    int k;
    cin >> k;

    for (int i = 0; i < k; i++) {
        int x, y;
        cin >> x >> y;
        obs[x][y] = true;
    }

    int ans = solve(st);

    if (ans == -1) {
        cout << "Impossible" << endl;
    }
    else {
        cout << ans << endl;
    }

    return 0;
```

}



四、题目 4（分支限界法，选做不占分）

小李希望开车到全国各地旅游，他发现旅途中各个城市的汽油价格也不相同，显然如果能够采取合理的加油方式，将会节省整个旅途的费用。假设汽车开始时油箱为空，汽车每走一个单位的距离将耗费一个单位的汽油。请帮助小李找到一种最省钱的方式完成他的整个自驾游。

输入要求：

输入的第一行包含两个整数 n（1 ≤ n ≤ 1000）和 m（0 ≤ m ≤ 10000），分别表示城市的数量以及道路的数量。其后的一行包含 n 个整数，分别表示 n 个城市的汽油价格。其后的 m 行，每一行包含三个整数 u,v,d，表示城市 u，v 之间的距离为 d（1 ≤ d ≤ 100）。最后一行包含三个整数 c（ 1 ≤ c ≤ 100）,s,e, 分别表示油箱的容量，出发的城市和最后到达的城市。

输出要求：

如果能够帮助小李找到一个最省钱从 s 到 e 的路线，则输出对应的费用，否则输出“impossible”。

样例输入：

4 6

8 10 6 7

0 1 8

0 3 12

1 2 10

2 3 7

1 3 9

0 2 10

10 0 3

要求：

1、画出采用分支限界法求解样例输入时的解空间树（必要时可以用省略号）。

2、给出采用分支限界法求解该问题时的目标函数，约束条件以及限界函数。

3、采用分支限界法编写程序实现上述题目要求。

基本思想：

分层图+最短路径

参考文档：

https://blog.csdn.net/qq_45735851/article/details/108219481?spm=1001.2101.3001.6650.5&utm_medium=distribute.pc_relevant.none-task-blog-2%7Edefault%7EBlogCommendFromBaidu%7Edefault-5-108219481-blog-124555794.pc_relevant_downloadblacklistv1&depth_1-utm_source=distribute.pc_relevant.none-task-blog-2%7Edefault%7EBlogCommendFromBaidu%7Edefault-5-108219481-blog-124555794.pc_relevant_downloadblacklistv1&utm_relevant_index=10